

深度学习作业 4

221900180 田永铭 tianym2022@smail.nju.edu.cn

2024 年 12 月 25 日

Problem 1 损失函数计算

已知一个线性分类器的预测得分为以下矩阵： $S = \begin{bmatrix} 4.8 & 2.5 & 2.0 \\ -3.1 & 0 & 12.9 \end{bmatrix}$

- (a) 其中每一行表示一个样本的预测得分，每一列表示一个类别。假设正确类别的索引为 $[0, 2]$ ，请计算 SVM 的损失函数值，公式如下： $L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$
- (b) 对于这个线性分类器的预测，计算出 softmax 概率以及交叉熵损失。

Solution: 解：

(a)

$$\begin{aligned} L &= \frac{1}{2} \sum_{i=1}^2 \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \frac{\max(0, 2.5 - 4.8 + 1) + \max(0, 2.0 - 4.8 + 1)}{2} + \frac{\max(0, -3.1 - 12.9 + 1) + \max(0, 0 - 12.9 + 1)}{2} \\ &= 0. \end{aligned}$$

所以 SVM 损失函数值为 0.

(b)

- 计算 softmax 概率：

根据公式：

$$P(y = k | x_i) = \frac{e^{s_k}}{\sum_{j=0}^2 e^{s_j}},$$

得：(注意我这里都保留了两位小数)

$$S_{\text{softmax}} = \begin{bmatrix} 0.86 & 0.09 & 0.05 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

- 计算交叉熵损失：

根据单样本的交叉熵损失公式 $L_i = -\log \frac{e^{y_i}}{\sum_j e^{s_j}}$ ，得：(注意我这里都保留了两位小数)

$$L_{\text{entropy}} = -\frac{\log \frac{0.86}{0.86+0.09+0.05} + \log \frac{1.00}{1.00+0.00+0.00}}{2} \approx 0.07.$$

Problem 2 numpy 实现带 L2 正则化的逻辑回归

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 # 生成高斯分布的二分类数据
6 def generate_data(n_samples=100, mean1=(2, 2), mean2=(-2, -2), cov=((1, 0)
7     , (0, 1))):
8     np.random.seed(42)
9     X1 = np.random.multivariate_normal(mean1, cov, n_samples) # 正类
10    X2 = np.random.multivariate_normal(mean2, cov, n_samples) # 负类
11    Y1 = np.ones((n_samples, 1))
12    Y2 = np.zeros((n_samples, 1))
13    X = np.vstack((X1, X2))
14    Y = np.vstack((Y1, Y2))
15    indices = np.arange(X.shape[0])
16    np.random.shuffle(indices)
17    return X[indices], Y[indices]
18
19 # Sigmoid 函数
20 def sigmoid(z):
21     return 1 / (1 + np.exp(-z))
22
23
24 # 逻辑回归 (带 L2 正则化)
25 def logistic_regression(X, Y, lr=0.01, epochs=1000, lambda_=0.1):
26     m, n = X.shape # 样本数量和特征数量
27     W = np.zeros((n, 1)) # 初始化权重
28     b = 0 # 初始偏置
29     losses = []
30
31     for epoch in range(epochs):
32         # 前向传播
33         Z = np.dot(X, W) + b # 线性部分
34         A = sigmoid(Z) # Sigmoid 激活
35         #todo 损失函数 (交叉熵 + L2 正则化项)
36         #loss = ?
37         losses.append(loss)
38
```

```
39     #todo 反向传播
40     #dW = ? 交叉熵 + L2 正则化 梯度
41     #db = ? 偏置的梯度
42
43     # 参数更新
44     W -= lr * dW
45     b -= lr * db
46
47     return W, b, losses
48
49
50 # 预测函数
51 def predict(X, W, b):
52     Z = np.dot(X, W) + b
53     A = sigmoid(Z) # 激活函数
54     return (A >= 0.5).astype(int)
55
56
57 # 绘制数据分布和决策边界
58 def plot_decision_boundary(X, Y, W, b):
59     plt.figure(figsize=(8, 6))
60     plt.scatter(X[Y.flatten() == 0][:, 0], X[Y.flatten() == 0][:, 1], color='
        red', label='Class 0')
61     plt.scatter(X[Y.flatten() == 1][:, 0], X[Y.flatten() == 1][:, 1], color='
        blue', label='Class 1')
62     x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
63     y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
64     xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max,
        0.01))
65     grid = np.c_[xx.ravel(), yy.ravel()]
66     Z = predict(grid, W, b)
67     Z = Z.reshape(xx.shape)
68     plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
69     plt.legend()
70     plt.title("Decision Boundary")
71     plt.show()
72
73
74 # 主程序
75 if __name__ == "__main__":
76     X, Y = generate_data(n_samples=100)
77     X = (X - np.mean(X, axis=0)) / np.std(X, axis=0) # 特征标准化
```

```

78 W, b, losses = logistic_regression(X, Y, lr=0.1, epochs=1000, lambda_=0.5)
79 print(f"最终损失: {losses[-1]}")
80 plt.plot(losses)
81 plt.title("Loss Curve")
82 plt.xlabel("Epochs")
83 plt.ylabel("Loss")
84 plt.show()
85 # 绘制决策边界
86 plot_decision_boundary(X, Y, W, b)

```

Solution: 解:

补充部分的代码如下:

```

1 loss = -(1 / m) * np.sum(Y * np.log(A) + (1 - Y) * np.log(1 - A))
2 loss += (lambda_ / (2m)) * np.sum(W ** 2) ## 除以2是方便梯度计算, 注意我这里
   已经除以2了, 后面梯度就不用除以2了, 根据老师所讲这里需要除以m
3
4 .....
5
6 dZ = A - Y # 损失对预测的梯度
7 dW = (1 / m) * np.dot(X.T, dZ) + lambda_ * W / m # 权重参数
8 db = (1 / m) * np.sum(dZ) # 偏置

```

关键步骤如下:

在逻辑回归中, 我们通过最小化损失函数来优化模型参数。损失函数由两部分组成: 交叉熵损失和 L2 正则化项。目标是计算损失函数关于模型参数 (权重 W 和偏置 b) 的梯度。

损失函数

逻辑回归的损失函数由交叉熵损失和 L2 正则化项组成:

$$\mathcal{L}(W, b) = -\frac{1}{m} \sum_{i=1}^m [Y^{(i)} \log(A^{(i)}) + (1 - Y^{(i)}) \log(1 - A^{(i)})] + \frac{\lambda}{2m} \sum_{i=1}^n W_i^2$$

为了通过梯度下降方法优化模型, 我们需要计算损失函数对于模型参数 W 和 b 的梯度。

(a) 对权重 W 的梯度

首先, 我们需要对损失函数关于权重 W 求梯度。通过链式法则, 我们可以得到:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

其中, $\frac{\partial \mathcal{L}}{\partial Z} = A - Y$ 是交叉熵损失对于 Z 的梯度, 这一步也是由链式法则直接计算得到的, 且:

$$\frac{\partial Z}{\partial W} = X^T$$

因此, 损失函数对于 W 的梯度为:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{m} \sum_{i=1}^m (A^{(i)} - Y^{(i)}) X^{(i)} + \lambda W / m$$

其中, $(A^{(i)} - Y^{(i)})$ 是交叉熵损失对预测值 $A^{(i)}$ 的梯度, λW 是 L2 正则化项对 W 的梯度。

(b) 对偏置 b 的梯度

损失函数对于偏置 b 的梯度可以通过类似的方法计算:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial Z} \cdot \frac{\partial Z}{\partial b}$$

其中, $\frac{\partial Z}{\partial b} = 1$, 所以损失函数对 b 的梯度为:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \sum_{i=1}^m (A^{(i)} - Y^{(i)})$$

在 Colab 平台运行, 最终结果损失为: 0.04380993306764013。结果见图 1。

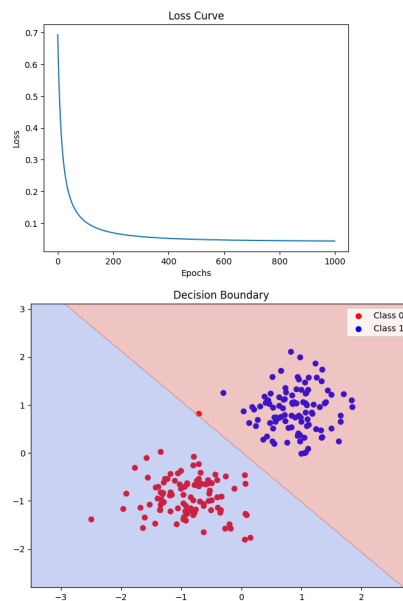


图 1: 训练结果图

Problem 3 L1 正则化和 L2 正则化的主要区别是什么？它们分别适用于什么场景？

Solution:

特征	L1 正则化
公式	$\lambda \sum_{i=1}^n w_i $
惩罚方式	权重的绝对值（L1 范数）
对权重的影响	产生稀疏解（部分权重趋近于零）
模型稀疏性	更倾向于产生稀疏模型（有些权重为零）
优化算法	产生不连续的梯度，可能导致优化不稳定

特征	L2 正则化
公式	$\lambda \sum_{i=1}^n w_i^2$
惩罚方式	权重的平方（L2 范数）
对权重的影响	使权重较小，但不完全为零
模型稀疏性	不倾向于产生稀疏模型，所有权重较小
优化算法	产生平滑的梯度，优化更稳定

L1 正则化适用于高维数据，用于特征的选择，或者需要稀疏模型时；L2 正则化适用于特征之间存在多重共线性时，避免过拟合，需要模型稳定训练时。

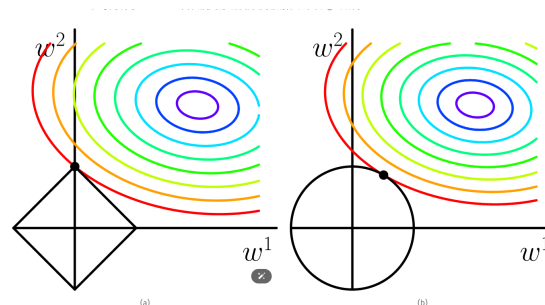


图 1 (a) ℓ_1 -ball meets quadratic function. ℓ_1 -ball has corners. It's very likely that the meet-point is at one of the corners. (b) ℓ_2 -ball meets quadratic function. ℓ_2 -ball has no corner. It is very unlikely that the meet-point is on any of axes.

图 2: L1 为什么稀疏

对于 L1 稀疏的理解可以用一张图来展示，如图 2。来源：Sparsity and Some Basics of L1 Regularization.

Problem 4 AdaGrad 和 RMSProp 的主要区别是什么？Adam 优化器结合了哪些优化算法的优点？它的主要超参数有哪些？

Solution: 解：

1. 主要区别：

AdaGrad 使用累计梯度的平方和用来控制学习率大小，这会导致学习率一直减小，一般凸优化问题中表现较好效果，而 RMSProp 使用指数加权移动平均 (EMA) 计算梯度的历史平方和，使得梯度平方的历史信息逐渐淡化，使算法更适合处理非平稳目标，往往更适合深度学习（以及大多数非凸问题），两者主要区别在代码中就体现为一行，见图 3

2. 结合哪些算法的优点：

Adam 优化器结合了 RMSProp 的自适应学习率，避免某些参数的梯度过大或过小，从而更稳定地更新参数的优点和 Momentum 的动量加速，避免梯度波动过大，有助于更快地找到最优解的优点

3. Adam 的主要超参数：

有 η 学习率 β_1 一阶动量衰减系数 β_2 二阶动量衰减系数，避免除 0 的小量 $\epsilon = 1e-7$ 。

■ AdaGrad (Adaptive Gradient Algorithm)

AdaGrad

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared += dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

图 3: AdaGrad 和 RMSProp 的主要区别

Problem 5 鞍点和局部最优值的区别是什么？如何通过 Hessian 矩阵的特征值来区分它们？

Solution: 解：

1. 区别：

局部最优值是指在某点 x^* 附近存在一个小的邻域，在该区域内的所有点的函数值 $f(x)$ 均不小于 (或不大于) $f(x^*)$ ，则 $f(x^*)$ 称为局部最优值。这些点梯度为 0.

鞍点是指函数在某点 x_{saddle} 的梯度为零或者很小，但该点既不是局部极大值也不是局部极小值。换句话说，函数值在该点沿某些方向是极小值，而沿某些方向是极大值。这样的点很像是马鞍的形状

2. 如何区分：

局部最优值的 Hessian 矩阵为正定或者负定矩阵 (特征值都正或者都负)，鞍点的 Hessian 矩阵的特征值有正有负，如果 Hessian 矩阵为半正定或者半负定矩阵，则需要进一步判断。